Obsidian ERP v4 — Live End-to-End Test Plan (section-by-section sign-off)

> **Purpose.** This is the canonical live test for the v4 ship — it supersedes `PHASE_2P_LIVE_RETEST.md`. It is organized by **business workflow**, not by code module, so you can see what real-world job each section proves the system can do. We run it **section by section**: you execute a section, capture feedback in its **Findings** block, we fix/hand off, you **sign off**, then we move to the next. Nothing advances on an unsigned section.
>
> **Pilot context.** The pilot is an Ethiopian **printing business** (the "Pana" company). Examples use printing work: products like *Business Cards*, *Banners*, *Brochures*; raw materials like *Art Paper 300gsm*, *Cyan Ink*, *Lamination Film*; a production run is a *print job*. Currency is **ETB**.
>
> **Assumes:** Phase 2P + 2P-FINAL (`docs/v4/PHASE_2P_FINAL_HANDOFF.md`) are built on `feat/v4-phase-2p-enterprise-ship`. Run the pre-flight (§0) once; then §A onward.

How to read each section

- **The business workflow** — what a real SME is doing and why it matters.
- **What the system automates** — what happens under the hood so you know what "correct" looks like.
- **Run it** — the click-path, performed as the stated role.
- **Expected** — the assertions that must hold.
- **Findings / Sign-off** — your notes + the gate to the next section.

---

§0 — Pre-flight (run once)

What the system automates. A brand-new tenant should go from empty to operational via the onboarding wizard alone — company, warehouses, manufacturing defaults, and the first users — with no Frappe desk visits.

Run it

1. Confirm static gates on the branch: `npx tsc --noEmit` (0 errors) and `npx vitest run` (the only expected red is the flaky `phase-2n` GlobalDashboard stub timeout).
2. Confirm env: `NEXT_PUBLIC_ERP_API_URL`, `ERP_API_KEY`, `ERP_API_SECRET` point at the Pana instance; the app and Frappe are **same-origin** (so the browser sends the `sid` cookie to `/api/**`). If not same-origin, set the cookie domain so `sid` is forwarded — **RBAC depends on this**.
3. Log in as **admin** (System Manager). Visit `/onboarding`. Walk all 5 steps: Company (Pana) → Operations (**run provision** → green check: creates Stores/WIP/FG warehouses + Manufacturing Settings) → Team (invite users, see §A) → Catalog (skip) → Done.
4. Seed the demo users with the role matrix below (via `/settings/users` or the onboarding Team step). Each user gets a welcome email / reset link; set their passwords.

User	Roles
`admin`	System Manager

User	Roles
`sales@pana.test`	Sales User, Sales Manager
`accounts@pana.test`	Accounts User, Accounts Manager
`stock@pana.test`	Stock User
`mfg@pana.test`	Manufacturing User, Stock User

Expected: onboarding completes; provision is idempotent (re-running is safe); all five users exist with the listed roles (reload `/settings/users` — roles persist).

Findings:

- …

Sign-off: ☐ Pre-flight complete — proceed to §A.

---

§A — RBAC & Roles (the access spine) · *ship gate*

The business workflow. A real company can't give every employee the keys to the accounting ledger. The salesperson quotes and sells; the accountant invoices and reconciles; the stock clerk receives and counts; the production lead runs jobs. Each should see their own work and be **unable** to touch the rest — not just in the UI, but at the data layer.

What the system automates. v4 forwards each user's Frappe session to the backend, so ERPNext's own role-permission engine decides what every API call may read or write. The UI's `<Can>` gating hides what you can't do; the server **enforces** it even if someone calls the API directly.

Run it (log in as each user in turn)

1. **admin** — sanity: every module loads, `/settings/users` shows the list + Invite. *(This is the regression bar: admin must see everything exactly as before.)*
2. **sales@** — CRM + Selling load. Try to open an accounting GL/Journal page or call its API directly → must be **denied (403)**, not shown.
3. **accounts@** — Accounting, Payments, Reports load. Try to edit a BOM or submit a Work Order → denied.
4. **stock@** — Stock Health, receive, count load. Try to post a Journal Entry or edit a price list → denied.
5. **mfg@** — Cockpit + Start/Finish job load. Try to open Accounting reports or `/settings/users` → denied.
6. **No session** — hit any `/api/**` CRUD route with cookies cleared → **401**.

Expected: the access matrix in §0 holds exactly. Denials are clean ("You do not have permission…"), never a stack trace or a silent empty table that's really a 403. `<Can>`-hidden buttons are also server-denied if forced.

Findings:

- …

Sign-off: ☐ RBAC enforced per role — proceed to §B.

---

§B — Lead-to-Cash (Selling) · run as `sales@`, invoice/pay as `accounts@`

The business workflow. A customer asks for 1,000 business cards. The shop captures the lead, quotes a price, confirms the order, delivers the cards, invoices, and gets paid. This is the revenue spine — every printing job that earns money runs through it.

What the system automates. Each step pre-fills the next from the last (Quotation → Sales Order → Delivery Note → Sales Invoice), carrying customer, items, prices, and the **back-links** that connect the documents. The Sales Invoice created from an order knows which order (and which delivery) it came from, so the **flow rail** on each document shows the whole chain lighting up stage by stage.

Run it

1. As `sales@`: **CRM** → **New Lead** ("Addis Printing Client"). Convert to **Customer**.
2. **New Quotation** for the customer: item *Business Cards*, qty 1000, a rate. Submit.
3. From the Quotation, **Create Sales Order** (prefilled). Submit. Open the SO → the **flow rail** shows Quotation (done) → Sales Order (current).
4. From the SO, **Create Delivery Note** (prefilled). Submit. The SO rail's Delivery stage lights up.
5. From the SO (or DN), **Create Sales Invoice**. Open the SI → its **Items carry** `sales_order` (and `delivery_note`); the SI total/taxes match what ERPNext computes (this is the `make-from` canonical path). The SO rail's **Invoice stage lights up with the SI name**; CrossFlow "View " resolves.
6. As `accounts@`: open the SI → **Create Payment Entry** (only shows while outstanding > 0). Pay in full. Reopen the SI → **no "Create Payment Entry" prompt**; the Payment stage shows the PE.

Expected: every stage of the rail resolves; no `DocType not found` 404 / `Field not permitted` 417 in the network log; SI↔SO are genuinely linked; a paid invoice never re-prompts for payment.

Findings:

- …

Sign-off: ☐ Lead-to-Cash verified — proceed to §C.

---

§C — Procure-to-Pay (Buying) · run as `stock@` (receive) + `accounts@` (bill/pay)

The business workflow. To print the cards, the shop needs paper and ink. It requests the materials, orders them from a supplier, receives the delivery into stock, gets the supplier's bill, and pays it.

What the system automates. A Material Request becomes a Purchase Order (prefilled); receiving the goods raises a Purchase Receipt that **increases on-hand stock automatically**; the supplier bill (Purchase Invoice) and payment chain link back to the PO. Receiving is one click — the operator never builds a stock document by hand.

Run it

1. **New Material Request**: *Art Paper 300gsm* qty 50, *Cyan Ink* qty 10. Submit.
2. From the MR, **Create Purchase Order** (prefilled). Submit.
3. As `stock@`: open the PO → **"Receive items"** → the **ReceiveMaterialsModal** lists ordered vs received per line. Receive in full (or partial — confirm partials work). Confirm → a **Purchase Receipt** is

created+submitted; target is the implicit **Stores** warehouse.
4. Confirm idempotency: reopen "Receive items" → it shows a "View existing" link for the receipt but still allows further partials.
5. As `accounts@`: from the PO, **Create Purchase Invoice** → submit → **Create Payment Entry** → pay. The PO rail shows Receipt + Invoice + Payment stages resolved.

Expected: on-hand for paper/ink rises by the received qty (verify in §E); receiving never asks the operator to pick a warehouse; the PO→PR→PI→PE chain resolves on the rail.

Findings:

- …

Sign-off: ☐ Procure-to-Pay verified — proceed to §D.

---

§D — Plan-to-Produce (Manufacturing) · run as `mfg@` · *the SME automation centerpiece*

The business workflow. Now the shop actually prints. It starts a print job for 1,000 business cards, the job consumes paper and ink, and finished cards land in stock ready to deliver. In raw Frappe this means BOMs, Work Orders, two kinds of Stock Entries, WIP vs FG warehouses, and backflush settings — far too much for an SME. v4 collapses it to **three verbs: Create job → Start job → Finish job.**

What the system automates. Creating a job looks up the product's default BOM and sets the warehouses implicitly (no warehouse questions). **Start job** moves the raw materials into work-in-progress in one click. **Finish job** consumes the materials (backflush) and receives the finished goods into stock — again one click. Shortfalls are detected and turned into a pre-filled Material Request instead of a dead end.

Run it

1. Ensure *Business Cards* has a **default BOM** (paper + ink). If missing, the Create-job path surfaces a guided "create a default BOM" link — follow it once.
2. `/manufacturing` **Cockpit** → **New job** → pick *Business Cards*, qty 1000, planned date → Confirm. A **Planned** card appears (a submitted Work Order, warehouses set implicitly).
3. **Start job** on the card. If stock is sufficient → one click moves materials to WIP, **zero warehouse/BOM questions**. The card moves to **In production**.
4. **Shortfall path** (optional but verify): if paper/ink is short, Start job opens a **pre-filled Material Request** (multiple items, comma-separated) — not a blank form, not an error.
5. **Finish job** on the card → finished *Business Cards* are received into FG stock; materials are consumed. The card moves to **Done**.
6. Cross-check §E: paper/ink on-hand dropped; *Business Cards* on-hand rose.

Expected: a non-expert can run a full job with only product + qty + two clicks. No raw Stock Entry, no warehouse picker, no BOM jargon in the happy path. A job created before provisioning would fail on warehouses — confirm the onboarding-provisioned tenant doesn't hit this.

Findings:

- …

Sign-off: ☐ Plan-to-Produce verified — proceed to §E.

---

§E — Inventory & Stock Health · run as `stock@`

The business workflow. The shop needs to know what's on hand, what's running low, and reconcile when a physical count differs from the system. Stockouts stop production; dead stock ties up cash.

What the system automates. A live **Stock Health** view shows on-hand with a status pill (In stock / Low / Out) computed from each item's reorder level. Low/Out items get a one-click **Reorder** (pre-filled Material Request). A friendly **Stock Count** writes a Stock Reconciliation draft (it does **not** auto-submit — a human approves the adjustment).

Run it

1. **Stock → Stock Health.** Confirm on-hand reflects §C receipts and §D consumption. Check the KPI "Low / Out" tile.
2. Find a Low/Out item → **Reorder** → a pre-filled MR opens. (Don't need to submit.)
3. **Stock count** → enter an item + warehouse + a counted qty different from system → it computes the difference and writes a **Stock Reconciliation draft** (not submitted). Confirm the draft exists and is editable before posting.

Expected: status pills are correct against reorder levels; Reorder pre-fills; Stock Count never auto-adjusts inventory.

Findings:

- …

Sign-off: ☐ Stock Health verified — proceed to §F.

---

§F — Financial Reporting · run as `accounts@`

The business workflow. The owner needs to know if the business is profitable (P&L), what it owns and owes (Balance Sheet), and who owes money / who is owed (AR/AP aging). These are the reports an Ethiopian SME shows the bank and the tax authority.

What the system automates. Reports build their period selector from the company's **real Fiscal Years** (not a hardcoded calendar). If no Fiscal Year covers the chosen date, instead of a raw red error the report shows a **guided dialog** with an "Open Fiscal Year settings" action — because the fix is config, not code.

Run it

1. **Reports → Profit & Loss.** It renders a real account tree inside the engraved UI, with a skeleton while loading. Switch FY/period → it refetches.
2. **Balance Sheet** — same: real tree, engraved UI, skeleton.
3. **AR / AP aging** — buckets (0-30/31-60/61-90/90+) reflect the §B/§C invoices.
4. **Missing-FY case:** pick a date with no Fiscal Year → the **guided dialog** appears with the "Open Fiscal Year settings" action — **not** a raw "FiscalYearError" string.

Expected: reports are real, styled, skeleton-guarded, FY-driven; the missing-FY path is guided, not raw.

Findings:

- …

Sign-off: ☐ Financial Reporting verified — proceed to §G.

---

§G — Dashboards (global + 6 module hubs) · run as `admin`

The business workflow. Each role wants a glanceable home: the owner sees the whole business; the salesperson sees the sales pipeline; the stock clerk sees what's low. A dashboard with no data or no actions is just decoration.

What the system automates. One `DashboardShell` drives the global home and all six module hubs from a per-module config: a KPI row, a module-appropriate chart, an alerts list, recent docs, and quick actions — every number from a real query, projections honestly labeled "Estimate."

Run it

1. **Global** `/dashboard` — trend charts + alert tiles + "Estimate" projections + quick-create buttons + module cards, populated by the §B–§E data.
2. Visit each hub and confirm it renders via the shell with KPIs + **its chart** + alerts + recent:
   - `/sales/dashboard` (sales trend area chart)
   - `/crm/dashboard` (leads vs converted)
   - `/buying/dashboard` (ordered vs received)
   - `/stock/dashboard` (top items / low-stock)
   - `/manufacturing/dashboard` (jobs by status)
   - `/accounting/dashboard` (AR/AP aging)
3. Confirm: real data (not zeros after §B–§E), skeletons while loading, **no "AI"/"Forecast" labels**, dual-theme + 375px + reduced-motion.

Expected: all six hubs are rich and actionable, not empty ModuleHub shells; charts render with OKLCH tokens.

Findings:

- …

Sign-off: ☐ Dashboards verified — proceed to §H.

---

§H — Cross-cutting polish & P2 channels

The business workflow. The product must feel premium and consistent everywhere, and the convenience channels (email a document, get a browser nudge) should work — though these are P2, not ship gates.

Run it

1. **Flow rails / CrossFlow** across SO, SI, PI, WO, PO detail pages: skeleton → content fast, **no 16-request storm** in the network tab, no Hooks-order crash, header-field links (SO→WO, MR→PO, PO→PR) resolve.

2. **Theme/responsive/motion** sweep on every new surface (Cockpit, CreateJob/Receive/StockCount modals, FinancialReportView, all dashboards, settings/users, settings/notifications, onboarding): dual-theme, 375px, reduced-motion, OKLCH only, no black borders.
3. **Email (P2):** as admin, trigger `/api/email/send` from a SI/PO/PE (or note it's route-only, UI follow-up). Recipient resolves from the doc's contact email.
4. **Push (P2):** `/settings/notifications` → enable browser push → permission prompt → a test notification deep-links to a doc.

Expected: rails are fast and correct; the UI bar holds everywhere; P2 channels work or are honestly noted as route-only.

Findings:

- …

Sign-off: ☐ Polish + P2 verified — **E2E COMPLETE** → **merge** `feat/v4-phase-2p-enterprise-ship` → **develop** → **ship to VPS.**

Sign-off ledger

Section	Workflow	Status	Date
§0	Pre-flight	☐	
§A	RBAC & Roles	☐	
§B	Lead-to-Cash	☐	
§C	Procure-to-Pay	☐	
§D	Plan-to-Produce	☐	
§E	Stock Health	☐	
§F	Financial Reporting	☐	
§G	Dashboards	☐	
§H	Polish + P2	☐	

> **Our loop per section:** you run it → drop notes in **Findings** → I triage (fix here if it's harness/doc/config, hand a tight fix to the mesh if it's app code) → you re-verify → **sign off** → next section. We do not skip ahead on an unsigned section.